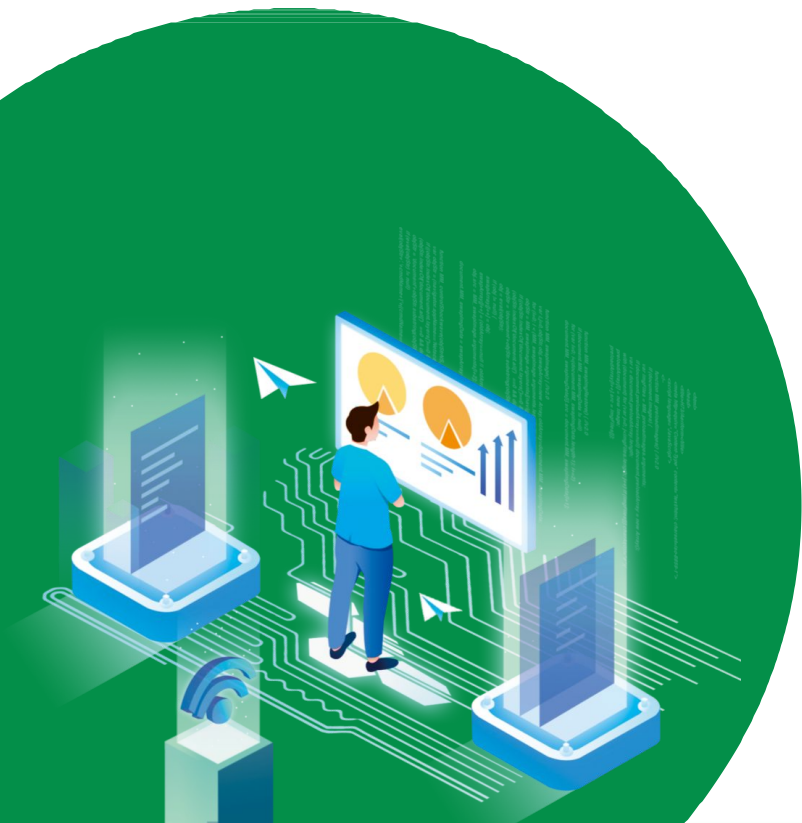


Pendle - Deep & Dive (Dev-mode)

Presenter: Link



CONTENT

01. Pendle features

02. Pendle Mechanics

03. Noodles-Pen onchain design

BEFORE START

Read Pendle-Pine-Noodles slide again

- ❖ **Yield-Bearing Assets (YBAs)**: Assets that automatically accumulate yield over time
- ❖ **Standardized Yield Token (SY)**: A yield-bearing asset (e.g., stSUI) is deposited and wrapped into a SY token
- ❖ **1 YBA = 1 SY-YBA**

Tokenizing Yield

- ❖ **Wrap**: YBA -> SY
- ❖ **Split**: SY -> PT + YT
 - **Principal Token (PT)**: Represents the underlying principal amount. Its value appreciates to par at maturity.
 - **Yield Token (YT)**: Represents the right to all future yield. Its value decays to zero at expiry.
- ❖ **Settle**: At maturity, PT is redeemable 1:1 for the underlying asset, and YT expires.

BEFORE START

Hieu: Can become LP

- Fees
- Incentive

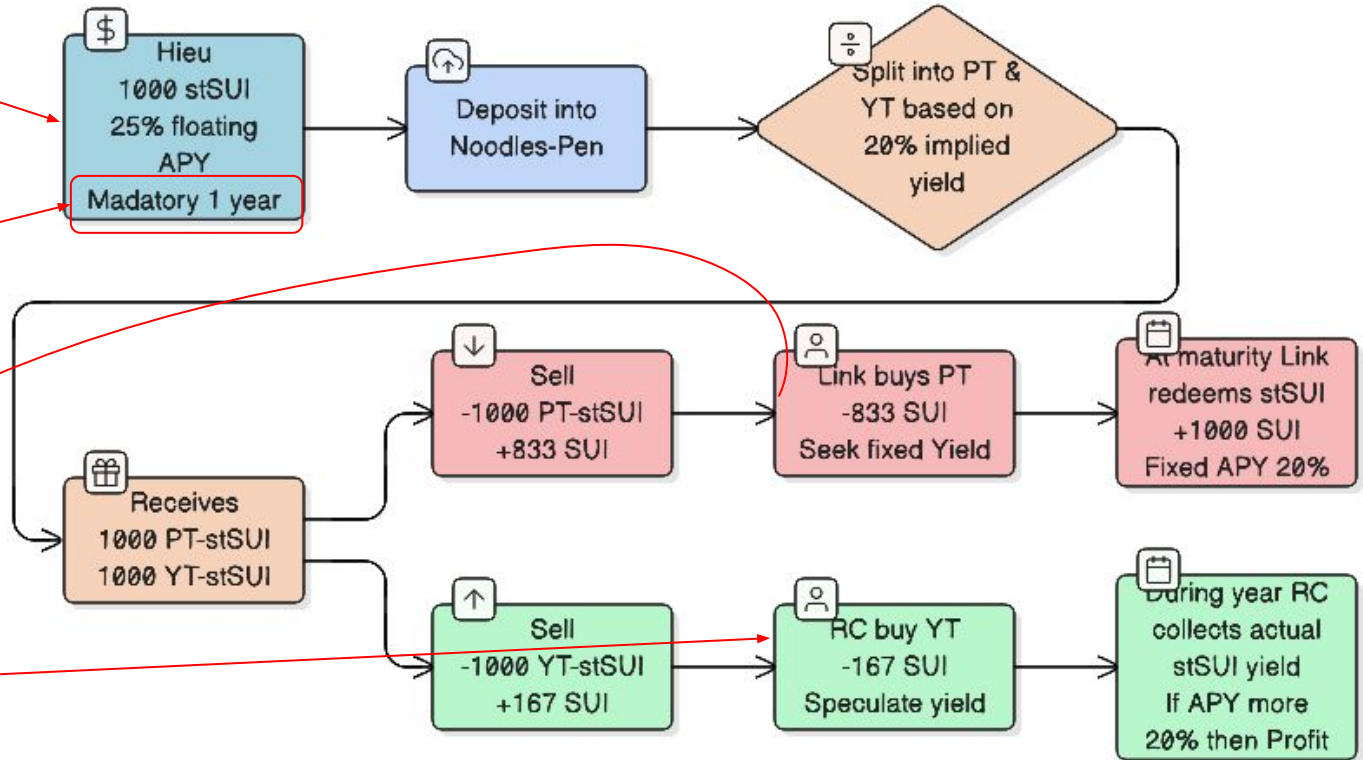
Market

Link: Short/Fixed Yield

- Fixed APY

Richard: Long Yield

- Earn future yield





1

Pendle Features



Pendle features: Market overview

Q1: Where is PT/YT Price????

Market

Buy YT

Buy PT

Current fixed APY

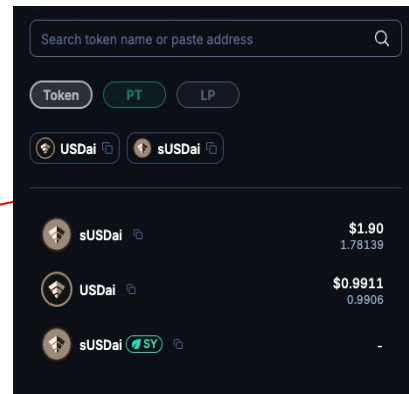
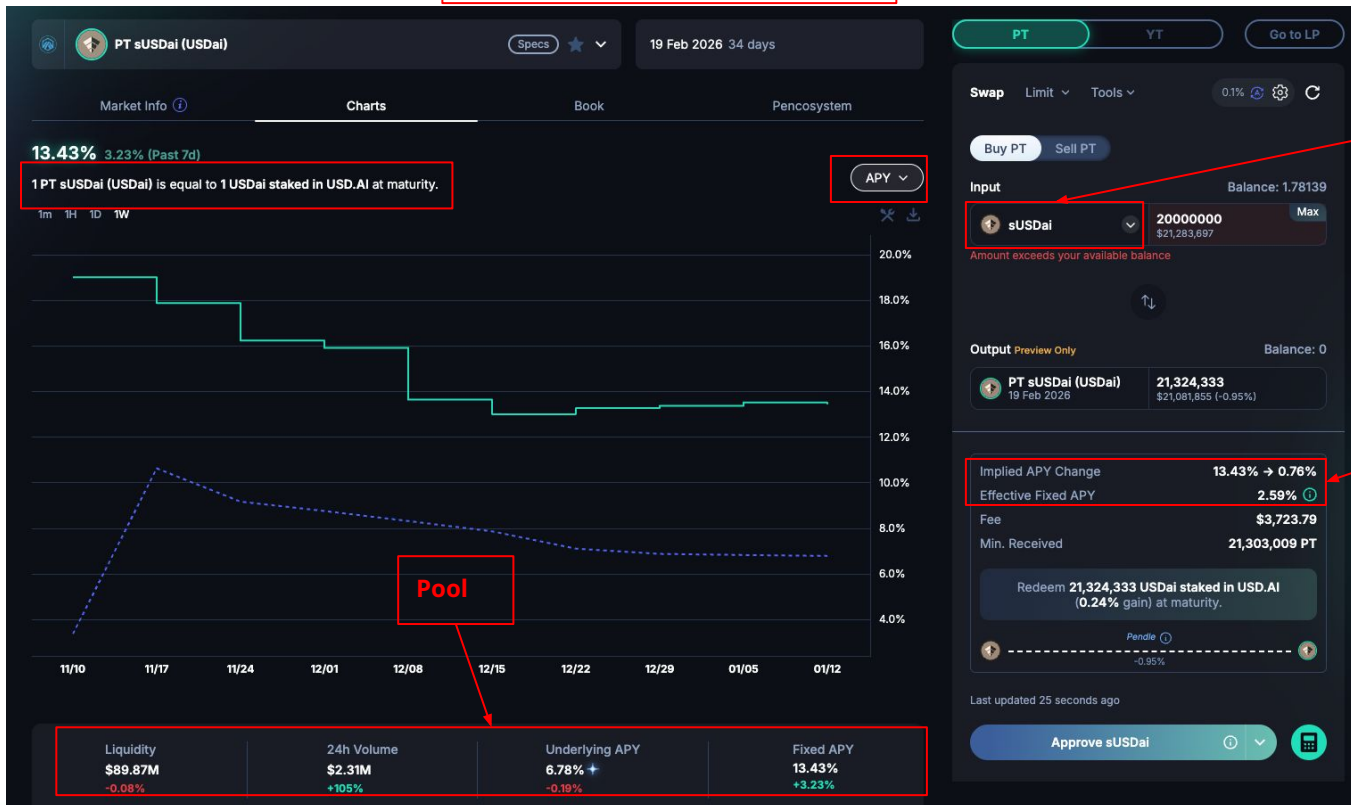
APY from sUSDe

Mandatory



Pendle features - Buy/sell PT

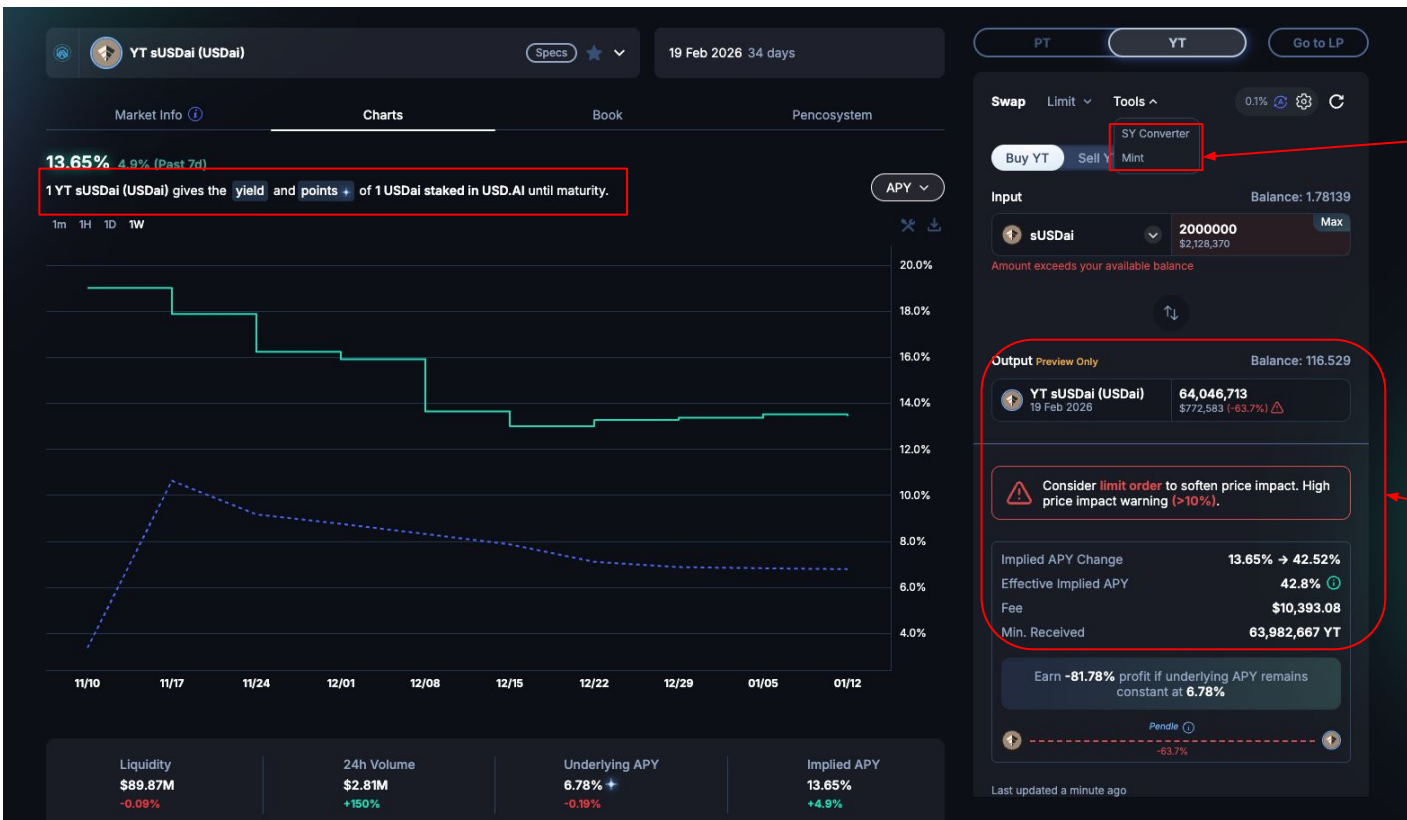
Pendle's design is APY-Oriented!



Price impact



Pendle features - Buy/sell YT



PT YT Go to LP

Swap Limit Mint

Mint Redeem

Tip: Minting PT and YT to sell one gives the same output as buying it directly with less gas fees.

Input Balance: 1.78139

sUSDai 0 \$0 Max

Output Balance: 0

PT sUSDai (USDai) 0 \$0

YT sUSDai (USDai) 0 \$0

Mint

PT YT Go to LP

Swap Limit Tools 0.1%

Buy YT Sell Y SY Converter Mint

Input Balance: 1.78139

sUSDai 2000000 \$2,128,370 Max

Amount exceeds your available balance

Output Preview Only Balance: 116.529

YT sUSDai (USDai) 64,046,713 \$772,583 (-63.7%)

Consider limit order to soften price impact. High price impact warning (>10%).

Implied APY Change 13.65% → 42.52%
Effective Implied APY 42.8%
Fee \$10,393.08
Min. Received 63,982,667 YT

Earn -81.78% profit if underlying APY remains constant at 6.78%

Pendle -63.7%

Last updated a minute ago

Price impact



Pendle features - LP

Liquidity \$89.87M

sUSDai SY \$67.42M (75.02%) \$22.45M (24.98%) PT sUSDai (USDai)

Est. Daily Pool Rewards 476.123 PENDLE

My Trade History				My Positions				Transactions			
Filter by wallet address				All Dates ▾ All Actions ▾ \$50 & above ▾				Trades Liquidity			
User	Action			Value	Time						
0x7269...eef9	Add Liquidity			\$1,000.93	18m						
0xf891...4c91	Add Liquidity			\$199.34	3h 55m						
0xe7a7...5936	Remove Liquidity			\$105.49	5h						
0x4f50...97ee	Remove Liquidity			\$1,500.00	5h 14m						
0xac56...f2ab	Add Liquidity			\$767.61	11h 40m						
0xef5...a3b4	Remove Liquidity			\$11,625.20	14h 32m						

Zap Manual Go to PT/YT

Add Liquidity Remove Liquidity 0.1% ⚙️ ↻

Underlying Input Balance: 1.78139

sUSDai 10000000 \$10,841,849 Max

Amount exceeds your available balance

PT Input Balance: 0

PT sUSDai (USDai) 3584699.0022203548 Max

Amount exceeds your available balance

Output Preview Only Balance: 0

LP sUSDai (USDai) 6,739,274 \$14,185,190 (0%) ⚙️

Pool Share 0% → 13.63% 7,704,880 vePENDLE

Implied APY Change 13.65% → 13.65%

Min. Received 6,735,897 LP

Last updated 5 seconds ago

Approve Tokens (2) ⓘ

Must be the same ratio with pool state

Incentive

Implied APY no change



Pendle features - Claim Yield & Rewards



Oxfb5...5df3

Updated <1 minute ago



Plasma Parade

Trade to earn extra XPL rewards!

My Current Balance

\$9.06

My Claimable Yield & Rewards

\$0.2858

Claim

Additional Rewards

Nothing to Claim



Arbitrum

\$9.06 (100.00%)



Ethereum

\$0 (0%)



Plasma

\$0 (0%)



HyperEVM

\$0 (0%)



BNB Chain

\$0 (0%)



Base

\$0 (0%)



Berachain

\$0 (0%)



Sonic

\$0 (0%)



Mantle

\$0 (0%)



Optimism

\$0 (0%)

My Positions

My Orders

Go to wallet address



All Assets

PT

YT

LP

USD

Underlying

Open Positions

Assets

Active Positions

Total Position Value

LP APY

Claimable Yield

Net P&L



weETH

25 Jun 2026 (160 days)

PT YT LP

0.001179

2.74%

(1.00X)

~0.061018

+0.051456

+0.12%



USDai

19 Feb 2026 (34 days)

PT YT LP

3.68332

-

0

-0.3261

-8.14%



sUSDai

19 Feb 2026 (34 days)

PT YT LP

1.40453

-

~0.2853

-0.3977

-18.75%

Claim yield & rewards



Pendle features - Conclusion

Pendle = Trading future yield

- **Price = interest rate**
- **APY-Oriented design**
- **Buy PT:** Short/fixed yield
- **Buy YT:** Long yield
- **Pendle pools:** PT/SY + time to maturity

Remember:

- Pendle never creates yield, yield always comes from external protocols
- Pendle only restructures cash flows

Noodles-Pen features

- **Yield Core:** Mint/redeem SY, PT, YT,...
- **Market AMM:** Buy/sell PT/YT/SY/YBA/Underlying asset
- **Pool:** Add/remove liquidity
- **Reward Manager:** Claim yield + rewards
- **\$NOODLES fee manager + incentive**



Pendle features - Question

sUSDe Ethena		2 Markets	\$42.27M \$532.45M	\$7.26M	328x	5.58%	⌵
Market ⌵		Liquidity ⌵ Total TVL	24h Volume ⌵	Underlying APY ⌵	YT Leverage ⌵	Fixed APY ⌵	
⬆️ sUSDe 07 May 2026 (111 days)		\$1.33M \$1.44M	\$11,377	4.78% ⬆️	YT 67x ⬆️	PT 5.04%	
⬆️ sUSDe 05 Feb 2026 (20 days)		\$40.94M \$531.01M	\$7.25M	4.78% ⬆️	YT 328x ⬆️	PT 5.58%	

Question 1: What does the market think about future yield in the sUSDe (Ethena)?

- A. The market thinks future yield to decrease
- B. The market thinks future yield to increase
- C. The market thinks future yield to remain stable

2

Pendle Mechanics





Yield Tokenization - SY

Why SY exists?

- Different YBAs have different mechanics:
 - Rebasing vs non-rebasing
 - Claim-based vs auto-compounding
- Without SY, AMM math would explode in complexity

ERC-5115: SY Token: <https://eips.ethereum.org/EIPS/eip-5115>

Provides a common yield interface:

- `deposit()` / `redeem()`
- `exchangeRate()` (or `syIndex`)
- `claimYield()` (optional)

Mathematically:

$SY_value = underlying_amount \times syIndex$



Yield Tokenization - Generic Yield Generating Pool

GYGP: A unified model for yield in DeFi

- Most *DeFi* yield mechanisms can be abstracted as a **Generic Yield Generating Pool**
- **Asset:** Unit of value for the pool. Total pool value: $TotalAsset(t)$
- **Shares:** Ownership units of the pool. Total supply: $TotalShares(t)$
- **Reward Tokens:** Pool may earn n reward tokens. Accumulated amount of token i at time t : $Reward_i(t)$
- **Exchange Rate:** $ExchangeRate(t) = TotalAsset(t) / TotalShares(t)$

User State

- For user u at time t :
 - $Assets_u(t) = Shares_u(t) \times ExchangeRate(t)$
 - $Rewards_{\{u,i\}}(t)$

Yield Accrual

- Pool earns ΔY assets at time t : $TotalAsset(t+) = TotalAsset(t) + \Delta Y$
- Exchange rate changes: $ExchangeRate(t+) = ExchangeRate(t) + \Delta Y / TotalShares(t)$
- Reward: $Reward_i(t+) = Reward_i(t) + \Delta Reward_i$

=> Once you model yield as GYGP, Pendle becomes a deterministic system!



Yield Tokenization - SY

```
1 interface IStandardizedYield {
2     event Deposit(
3         address indexed caller,
4         address indexed receiver,
5         address indexed tokenIn,
6         uint256 amountDeposited,
7         uint256 amountSyOut
8     );
9
10
11     event Redeem(
12         address indexed caller,
13         address indexed receiver,
14         address indexed tokenOut,
15         uint256 amountSyToRedeem,
16         uint256 amountTokenOut
17     );
18
19     function deposit(
20         address receiver,
21         address tokenIn,
22         uint256 amountTokenToDeposit,
23         uint256 minSharesOut,
24         bool depositFromInternalBalance
25     ) external returns (uint256 amountSharesOut);
26
27     function redeem(
28         address receiver,
29         uint256 amountSharesToRedeem,
30         address tokenOut,
31         uint256 minTokenOut,
32         bool burnFromInternalBalance
33     ) external returns (uint256 amountTokenOut);
34
35     function exchangeRate() external view returns (uint256 res);
36
37     function getTokensIn() external view returns (address[] memory res);
38
39     function getTokensOut() external view returns (address[] memory res);
40
41     function yieldToken() external view returns (address);
42
43     function previewDeposit(address tokenIn, uint256 amountTokenToDeposit)
44         external
45         view
46         returns (uint256 amountSharesOut);
47
48     function previewRedeem(address tokenOut, uint256 amountSharesToRedeem)
49         external
50         view
51         returns (uint256 amountTokenOut);
52
53     function name() external view returns (string memory);
54
55     function symbol() external view returns (string memory);
56
57     function decimals() external view returns (uint8);
58 }
```

SY-function

ERC-20



Yield Tokenization - Mint PT/YT

TRANSACTION ACTION
Call Mint Py From Token Function by [0xfB51f790...815085Df3](#) on [Pendle: RouterV4](#)

Transaction Hash: [0x86c238f2473ab5f8139057e4f246a4798d287892ed1a7bc84406c35ccc78fb45](#)

Status: Success

Block: [416744932](#) 101396 L1 Block Confirmations

Timestamp: 14 days ago (Jan-01-2026 10:29:02 AM +UTC)

From: [0xfB51f7904De1b68DE4392bea3207C5C815085Df3](#)

Interacted With (To): [0x888888888889758F76e7103c6CbF23ABbF58F946](#) (Pendle: RouterV4)

ERC-20 Tokens Transferred: 5

All Transfers **Net Transfers**

- From [0xfB51f790...815085Df3](#) To [Pendle: RouterV4](#) For 1 USDai (USDai)
- From [Pendle: RouterV4](#) To [Pendle: SY-USDai Token](#) For 1 USDai (USDai)
- From [Null: 0x000...000](#) To [0xdB795Dc5...01F851190](#) For 1 SY-USDai (SY-USD...)
- From [Null: 0x000...000](#) To [0xfB51f790...815085Df3](#) For 1 ERC20 ***
- From [Null: 0x000...000](#) To [0xfB51f790...815085Df3](#) For 1 ERC20 ***

Minting:

1. User deposits YBA
2. YBA wrapped into SY
3. SY $\rightarrow$ PT + YT

Redeeming (Post-Maturity)

- 1 PT $\rightarrow$ 1 SY $\rightarrow$ 1 YBA
- YT $\rightarrow$ 0

Redeeming (Pre-Maturity)

- Requires recombining
PT + YT $\rightarrow$ SY



Yield Tokenization - Mint PT/YT

```
/// @notice For details on the parameters (input, guessPtSwapToSy, limit, etc.), please refer to IPAllActionTypeV3.
function mintPyFromToken(address receiver, address YT, uint256 minPyOut, TokenInput calldata input)
    external
    payable
    returns (uint256 netPyOut, uint256 netSyInterm)
{
    address SY = IPYieldToken(YT).SY();

    netSyInterm = _mintSyFromToken(YT, SY, 0, input);
    netPyOut = _mintPyFromSy(receiver, SY, YT, netSyInterm, minPyOut, false);

    emit MintPyFromToken(msg.sender, input.tokenIn, YT, receiver, input.netTokenIn, netPyOut, netSyInterm);
}

/// @notice For details on the parameters (input, guessPtSwapToSy, limit, etc.), please refer to IPAllActionTypeV3.
function redeemPyToToken(address receiver, address YT, uint256 netPyIn, TokenOutput calldata output)
    external
    returns (uint256 netTokenOut, uint256 netSyInterm)
{
    address SY = IPYieldToken(YT).SY();

    netSyInterm = _redeemPyToSy(SY, YT, netPyIn, 1);
    netTokenOut = _redeemSyToToken(receiver, SY, netSyInterm, output, false);

    emit RedeemPyToToken(msg.sender, output.tokenOut, YT, receiver, netPyIn, netTokenOut, netSyInterm);
}
```



3

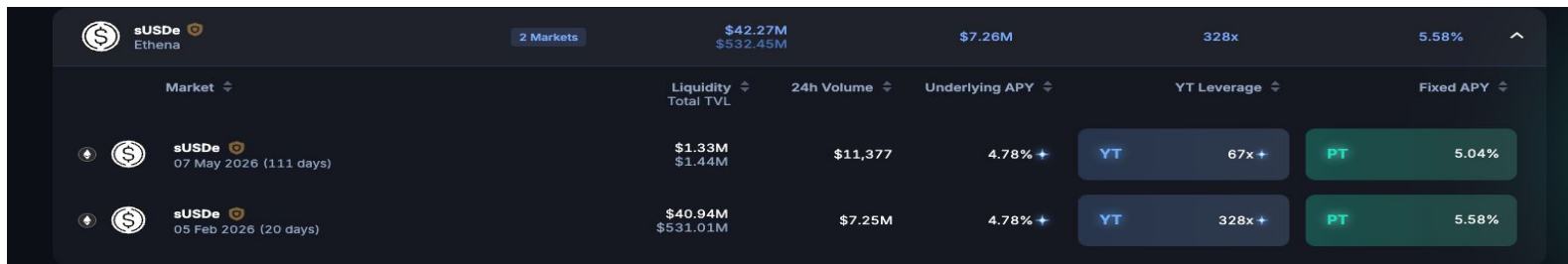
Liquidity Engines

Liquidity Engines

Pendle supports interest-rate trading via:

- **Primary Market:** AMM (Core focus)
- **Secondary Market:** Orderbook (Boros, for advanced traders seeking non-AMM execution)

Key difference from Uniswap: Traders are trading *implied yield*, not *spot price*



The screenshot shows the Pendle interface for sUSDe. At the top, it displays '2 Markets' with a total TVL of \$42.27M (\$532.45M) and a 24h volume of \$7.26M. Below this, a table lists two markets:

Market	Liquidity Total TVL	24h Volume	Underlying APY	YT Leverage	Fixed APY
sUSDe (07 May 2026 (111 days))	\$1.33M \$1.44M	\$11,377	4.78% +	67x +	5.04%
sUSDe (05 Feb 2026 (20 days))	\$40.94M \$531.01M	\$7.25M	4.78% +	328x +	5.58%

Question: Assume there are two markets, sUSDe-07-May-2026 and sUSDe-05-Feb-2026, with the same liquidity. Currently, the PT price in both markets is: 1 PT-sUSDe = 0.9 USDe

Which PT should you buy?

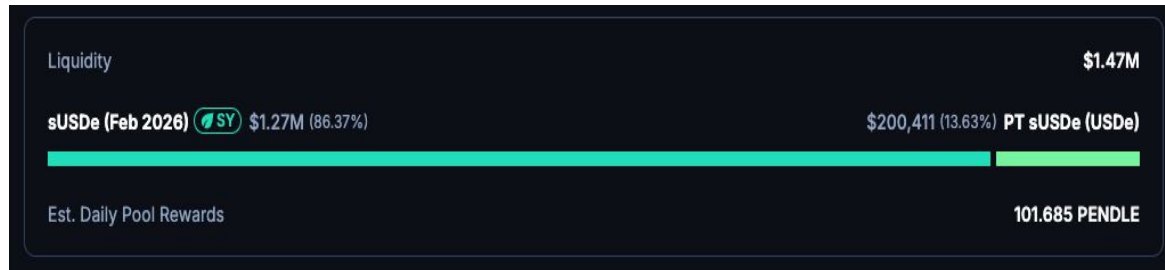
- A. sUSDe-07-May-2026
- B. sUSDe-05-Feb-2026
- C. Both markets give the same result



Liquidity Engines - AMM Design

Pool composition:

- **PT** ↔ **SY**



Why not PT ↔ YT?

- Yield already embedded in SY via *syIndex*
- YT payoff is path-dependent and decays to 0

Key design goals:

- Encode **interest rate trading** instead of spot price trading
- Ensure **smooth convergence** of PT price → 1 as maturity → 0
- AMM curve changes shape as time passes

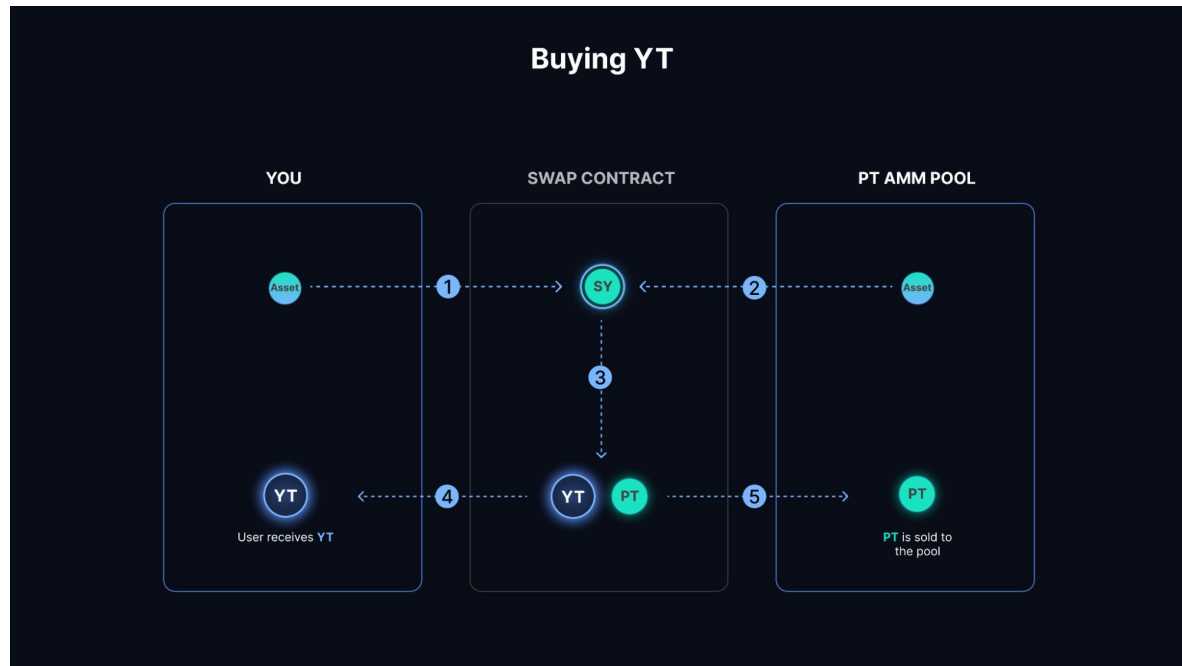
Core challenge: How do we build an AMM that efficiently trades interest rates, not just tokens?



AMM - Flash swaps

Buying YT:

- Buyer sends SY into the swap contract (auto-routed from any major token)
- Contract withdraws more SY from the pool
- Mint PTs and YTs from all of the SY
- Send the YTs to the buyer
- The PTs are sold for SY to return the amount from step 2





AMM - Flash swaps



TRANSACTION ACTION

Swap 2 sUSDai for 116.53 ERC20 *** on Pendle

Transaction Hash: 0x8f484ef62f8759336aead256fd0c922e03c9c4da97c9f9a21aa0dbefce91193d

Status: Success

Block: 416069989 116928 L1 Block Confirmations

Timestamp: 16 days ago (Dec-30-2025 11:39:49 AM +UTC)

From: 0xfB51f7904De1b68DE4392bea3207C5C815085Df3

Interacted With (To): 0x888888888889758F76e7103c6CbF23ABbF58F946 (Pendle: RouterV4)

ERC-20 Tokens Transferred: 7

All Transfers Net Transfers

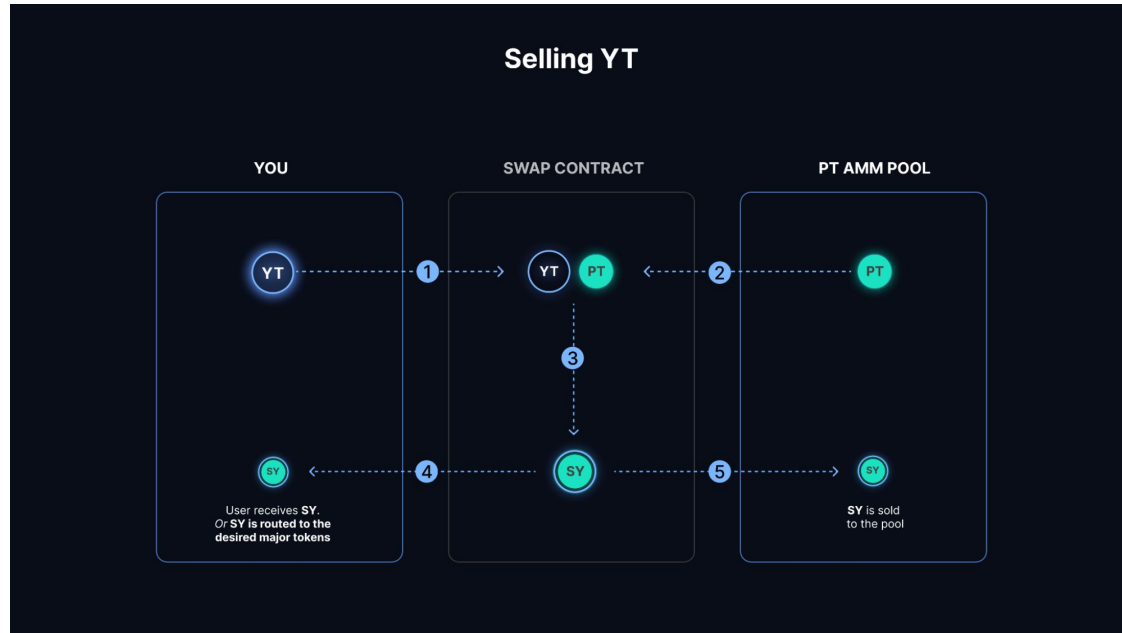
- From 0xfB51f790...815085Df3 To Pendle: RouterV4 For 2 Staked USDai (sUSDai)
- From Pendle: RouterV4 To Pendle: SY-sUSDai Tok... For 2 Staked USDai (sUSDai)
- From Null: 0x000...000 To 0x214D1950...1eF1bb66F For 2 SY-sUSDai (SY-sUS...)
- From 0x2092Fa5d...45413183E To 0x214D1950...1eF1bb66F For 107.87322412578248372 SY-sUSDai (SY-sUS...)
- From 0x2092Fa5d...45413183E To Pendle: Treasury For 0.020345618001435913 SY-sUSDai (SY-sUS...)
- From Null: 0x000...000 To 0xfB51f790...815085Df3 For 116.529406364589988848 ERC20 ***
- From Null: 0x000...000 To 0x2092Fa5d...45413183E For 116.529406364589988848 ERC20 ***



AMM - Flash swaps

Selling YT:

- Seller sends YT into the swap contract
- Contract borrows an equivalent amount of PT from the pool
- The YTs and PTs are used to redeem SY
- SY is sent to the seller (or routed to any major tokens, e.g. ETH, USDC, wBTC, etc)
- A portion of the SY is sold to the pool for PT to return the amount from step 2



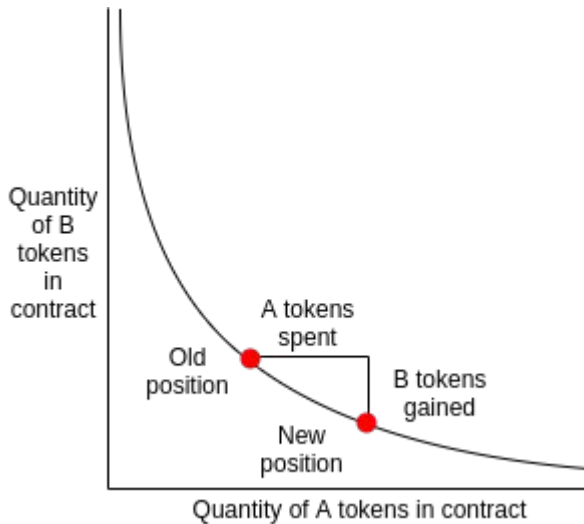


AMM - Curve

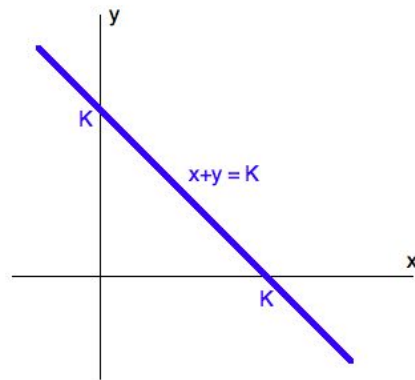
$$F(x,y) = x*y = k$$

How AMMs define price:

- An AMM defines a relationship between pool balances:
 - $F(x, y) = k$
 - x = asset (or SY)
 - y = PT
- A trade moves the pool along the curve $F(x, y) = k$
- Price = slope of the AMM curve (Math: $price = -dy/dx$)



$$x+y = k$$





Candidate 1: Geometric Mean

Model

$$x^{w_x} \times y^{w_y} = k$$
$$w_x + w_y = 1$$

- x = asset (or SY)
- y = PT
- price = $(w_x / w_y) \cdot (y / x)$

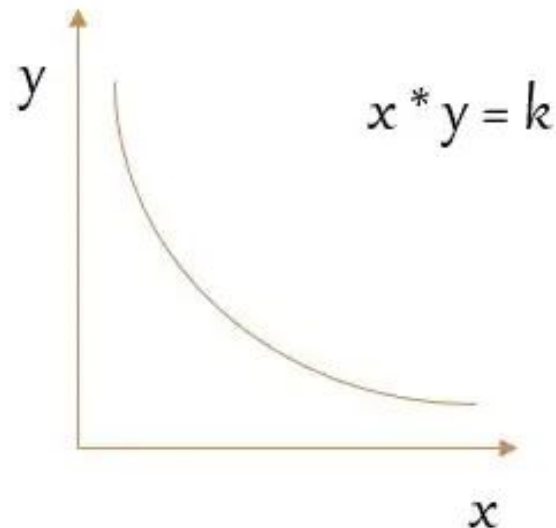
Idea for Pendle: Let weights change over time:

- Early: $w_x = w_y = 0.5$
- Near expiry: $w_y \rightarrow 1$

Problems:

- Price depends on token ratios, not time-to-expiry
- No exponential discounting (not bond-like)
- Hard to interpret price as an interest rate

=> **Not suitable for trading yield!**





Candidate 2: YieldSpace (Pendle V1)

Model $x^{1-t} + y^{1-t} = k$

$$p = \frac{y}{x + y}$$



$$price_a = \left(\frac{p}{1-p} \right)^t$$

- $t \in (0, 1)$ is normalized time to expiry
- *price (asset price in PT) = $-dy/dx$*
- **proportion**: p is a measure of the proportion of principal tokens in the pool

Strengths:

- Encodes time-to-expiry directly
- Bond-like behavior
- Near expiry $\rightarrow$ curve becomes very steep

Limitation:

- Liquidity is spread over all interest rates: From -100% to $+\infty$
- LP capital is locked providing liquidity where no one trades.

$\Rightarrow$ Capital inefficient for real interest rate markets (eg: 3% - 30%)

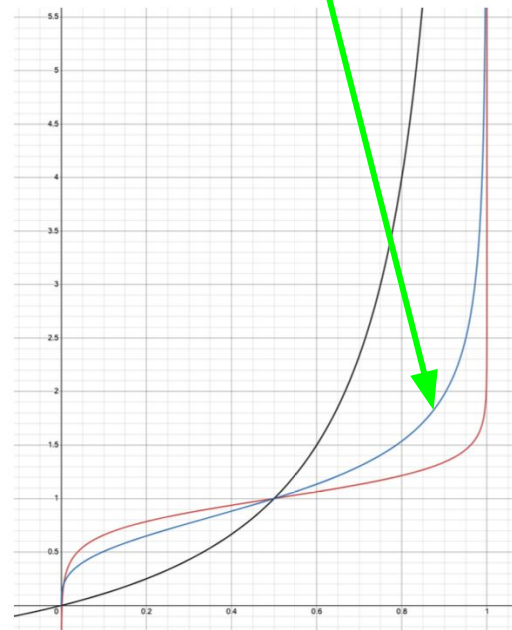


Figure 1: Price over proportion



AMM - Pendle Design

Key Insight: Yield Trading $\neq$ Token Trading

- Traders think in **interest rates**, not PT/SY ratios.
- Real markets only trade in a **narrow rate range**
 - Market cares about: 4% – 6% APY
 - Almost no volume outside this range

Pendle V2 Design Philosophy:

- Instead of defining an invariant and deriving price, define price (interest rate) directly as a function of pool state.
- Key Design Goals:
 - Trade interest rates directly
 - Concentrate liquidity where the market trades
 - Maintain smooth arbitrage and bond-consistent pricing



Pendle V2 AMM - Notional-style

Model

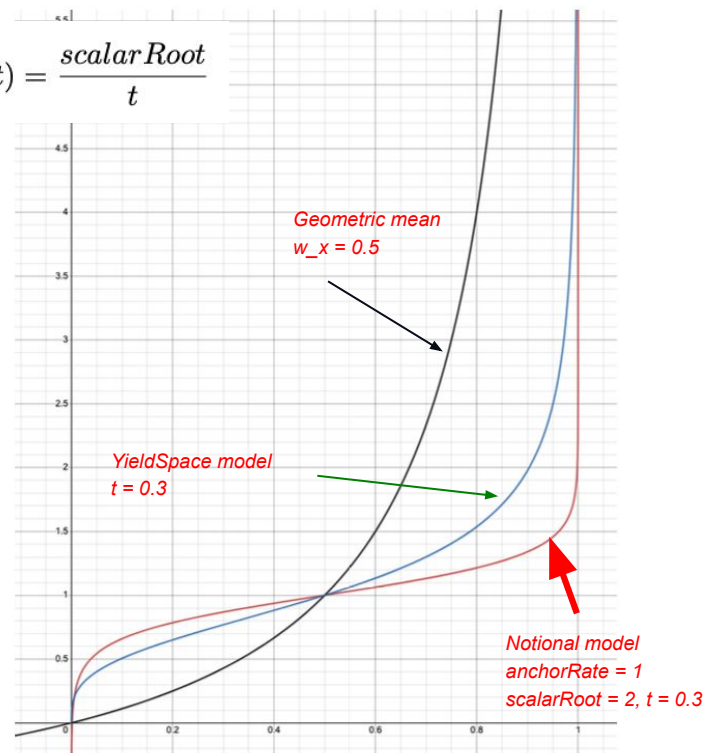
$$price_a(t) = \frac{\ln\left(\frac{p(t)}{1-p(t)}\right)}{rateScalar(t)} + rateAnchor(t)$$

$$rateScalar(t) = \frac{scalarRoot}{t}$$

- **rate scalar:** $rateScalar(t)$ at time t is a parameter used in calculating the price at time t , to adjust the capital efficiency
 - $rateScalar(t)$ = How wide the liquid trading range is
- **rateAnchor(t):** is a parameter used to adjust the interest rate around which the trading will be the most capital efficient, at time t
 - $rateAnchor(t)$ = Where liquidity is centered (e.g. 5% APY)

Strengths:

- Encodes time-to-expiry directly
- Trades around rateAnchor: Low slippage
- Customize for each YBA



(Price over Proportion)



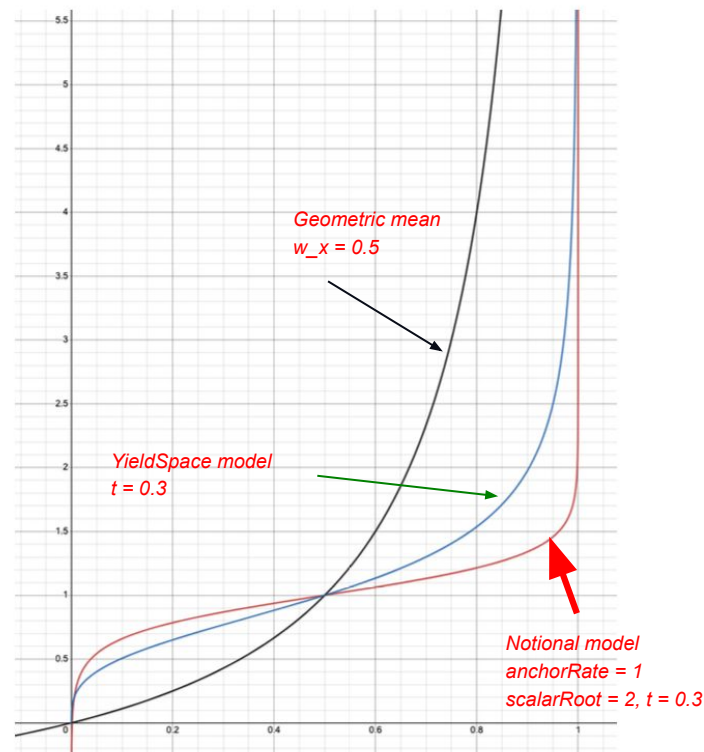
Pendle V2 AMM - Example

Setup

- Pool liquidity: 1,000,000 SY
- Time to expiry: 1 year
- Market anchor: 5% APY
- 4% APY $\rightarrow$ price ≈ 0.9615 , 6% APY $\rightarrow$ price ≈ 0.9434

Compare models:

- **YieldSpace:**
 - Liquidity spread across: $-100\% \rightarrow +\infty$ APY
 - Effective liquidity in 4–6% range: $\sim 2\%$ of total capital
- **Pendle V2:**
 - Liquidity concentrated around 5% APY
 - Effective liquidity in 4–6% range: $\sim 60\text{--}80\%$ of total capital



(Price over Proportion)



Pendle V2 AMM - Conclusion

What the Trader Is Really Doing?

- Trader inputs: SY and PT
- Under the hood: Each trade corresponds to a specific interest rate

Example:

- Swap 1 SY $\rightarrow$ 0.95 PT
- Implied rate $\approx$ 5.26% APY

Formula:

- **$\text{exchangeRate}(p) = \text{price}_a(t)$**
- **$\text{impliedRate} = \text{exchangeRate}^{(1 / \text{yearsToExpiry})}$**

$$\text{price}_a(t) = \frac{\ln\left(\frac{p(t)}{1-p(t)}\right)}{\text{rateScalar}(t)} + \text{rateAnchor}(t)$$



Pendle V2 AMM - Question

Question 1: If the current YT price is 0.05 SUI, and there are 180 days to expiry, what is the approximate Implied APY?

- A. 5.0%
- B. 10.9%
- C. 21.6%
- D. 50.0%

Question 2: In a Pendle market, a user sees a very high Fixed APY (e.g. 50%+) displayed on the interface. Which statement is MOST accurate?

- A. The user can lock a large amount of capital at this APY with minimal slippage
- B. A high Fixed APY usually means the pool has deep liquidity
- C. A high Fixed APY guarantees that buying PT will always outperform holding SY
- D. A high Fixed APY can coexist with very large price impact for non-trivial trade sizes



Fee & Rewards manager

Fee Structure

- YT Fee: 5% of all yield accrued by YT.
- Swap Fee: A percentage of the PT swap value, scaled by maturity. Distribution:

Swap Fees: 20% to LPs, 80% to vePENDLE holders who voted for the pool.

YT Fees: 80% to vePENDLE holders, 20% to the treasury/operations.

vePENDLE (Vote-Escrowed PENDLE)

Users lock PENDLE for up to 2 years to receive vePENDLE.

- Governance: Direct PENDLE emissions to specific pools via weekly gauge voting.
- Revenue Sharing: Earn protocol fees (YT and Swap fees).
- LP Boost: Increase PENDLE rewards for liquidity positions by up to 2.5x



Liquidity Provider

LPs in Pendle $\neq$ LPs in Uniswap

Pendle LPs provide liquidity to:

PT $\leftrightarrow$ SY

By doing so, LPs are:

- Market making on **interest rates**
- Exposed to **time** and **yield dynamics**

This is closer to a **fixed-income market maker** than a spot DEX LP



Liquidity Provider

Pendle LP returns come from multiple components:

1. **Swap fees**
 - From PT ↔ SY trades
2. **Implied rate convergence**
 - PT price naturally converges to 1 at maturity
3. **External incentives**
 - PENDLE emissions
 - Boosted by vePENDLE

Total LP PnL:

$\text{PnL} = \text{Fees} + \text{RateConvergence} + \text{Incentives}$



Liquidity Provider Risks

In Uniswap:

- Main risk = **Impermanent Loss** due to price divergence

In Pendle, IL is **not the primary risk**.

Key risks for Pendle LPs:

1. **Duration Risk**
 - Sensitivity to interest rate changes
2. **Yield Volatility Risk**
 - Changes in realized or expected yield
3. **Curve Exposure Risk**
 - AMM curve steepness depends on time to maturity



Liquidity Provider Risks

When is Pendle LP Attractive?

Pendle LP is most attractive when:

- Yield expectations are stable
- Realized yield is predictable
- Pool is close to maturity (low duration)

Less attractive when:

- Long-dated pools
- Macro yield uncertainty is high



APY Calculation

Key variables:

- `syIndex`
- `prevSyIndex`
- `timeElapsed`

Observed yield growth:

$$\text{RealizedYield} = \text{syIndex} / \text{prevSyIndex} - 1$$

Annualized:

$$\text{RealizedAPY} = (\text{syIndex} / \text{prevSyIndex})^{(1 / \tau)} - 1$$

- **Implied APY**
 - Derived from PT price
 - Forward-looking, market expectation
- **Realized APY**
 - From actual yield accrual
 - Backward-looking

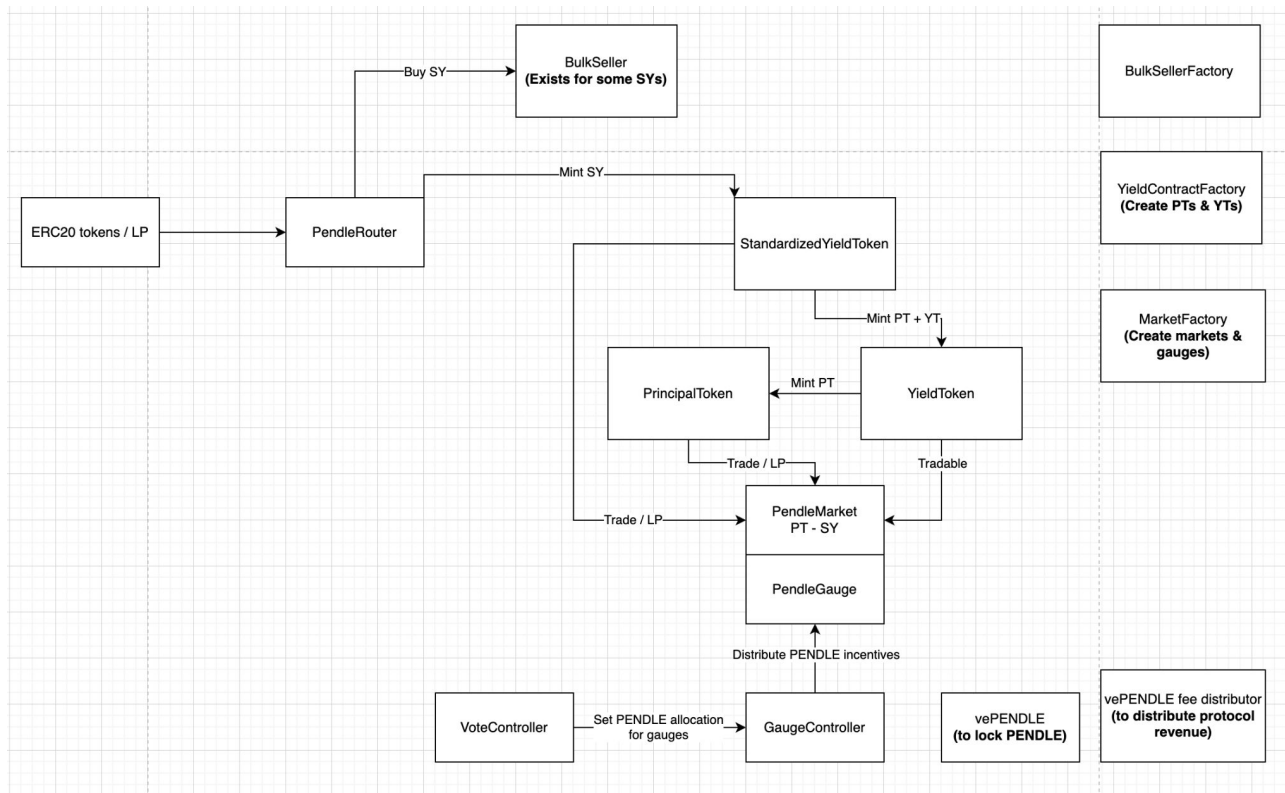
3

Noodles-Pen Onchain Design





Pendle High level design





Required Contract on SUI (Proposed)

Core Modules for Sui

1. **Standardized Yield** (`sy.move`): Defines the `SY<phantom T>` struct. It wraps native Sui yield assets (like staked SUI) into the SY standard.
2. **Yield Tokenization** (`yt_logic.move`): Manages the minting/burning of `PT<T>` and `YT<T>` coins. It utilizes `TreasuryCap` to control minting rights .
3. **Market Engine** (`market.move`): A Shared Object containing `Balance<PT>` and `Balance<SY>`. It implements the logit curve math using fixed-point libraries.
4. **Governance** (`ve_noodles.move`): Uses "Locked Objects" where users lock NOODLES objects to receive voting power.



Required Contract on SUI (Proposed)

Core contract:

- YieldWrapper
- SY
- PT
- YT
- Market
- MarketFactory
- Router
- FeeCollector
- Governance / veToken

Core components:

- SY Contract (EIP-5115-like)
- PT / YT contracts
- Market (AMM)
- Router
- Fee & Reward Manager



Question

Question: Why is `syIndex` critical to Pendle's design?

- A. It tracks token price
- B. It standardizes yield accrual
- C. It sets swap fees
- D. It replaces an oracle



References

- Pendles Finance v2 Docs:
<https://docs.pendle.finance/pendle-v2/ProtocolMechanics/Glossary> (Highly recommended)
- Pendles V2 Academy: <https://pendle.gitbook.io/pendle-academy/>
- Pendles Finance V2 source:
<https://github.com/pendle-finance/pendle-core-v2-public/tree/main> (Deep and dive)

Thank
you

